

软件详细设计说明书

1 引言

1.1 编写目的

本文档旨在对“生活助手平台”的软件设计进行详细说明。在《软件概要设计说明书》的基础上，本文档进一步细化系统的模块划分、功能逻辑、界面交互、异常处理及安全策略，为后续的编码实现、单元测试和系统集成提供直接、具体的依据。

预期读者包括项目开发人员（用于指导编码实现）、测试人员（用于设计测试用例）以及项目维护人员（用于理解系统内部逻辑）。

1.2 背景

- a. 待开发软件系统的名称：生活助手平台
- b. 任务提出者：课程项目组
- 开发者：课程项目组成员
- 用户：普通消费者、平台商家、配送人员及平台管理员
- 运行该软件的计算站（中心）：校园实验服务器或普通云服务器，客户端为个人计算机浏览器。

1.3 术语表

术语/缩写	解释
JWT	JSON Web Token，一种用于身份认证的紧凑、URL安全的令牌。
RESTful	一种基于HTTP协议的软件架构风格，本系统接口遵循其设计规范。
Bcrypt	一种自适应哈希加密算法，用于安全存储用户密码。
DTO	数据传输对象，用于在不同层（如控制层与服务层、前端与后端）之间封装数据。
DAO/Repository	数据访问对象，负责与数据库进行交互的持久层组件。
全局异常处理器	一种集中处理应用程序中未捕获异常的机制，返回统一的错误响应格式。
乐观锁	一种并发控制策略，假设数据冲突不频繁，通过在更新时检查版本号来保证数据一致性。

1.4 参考文档

- 《生活助手平台软件开发计划书（SDP）》V1.0
- 《生活助手平台软件需求规格说明书（SRS）》V1.0
- 《生活助手平台软件概要设计说明书》V1.0
- 《生活助手平台数据库设计说明书》（待编制）
- 《软件工程》课程实验指导书

2 系统结构设计及子系统划分

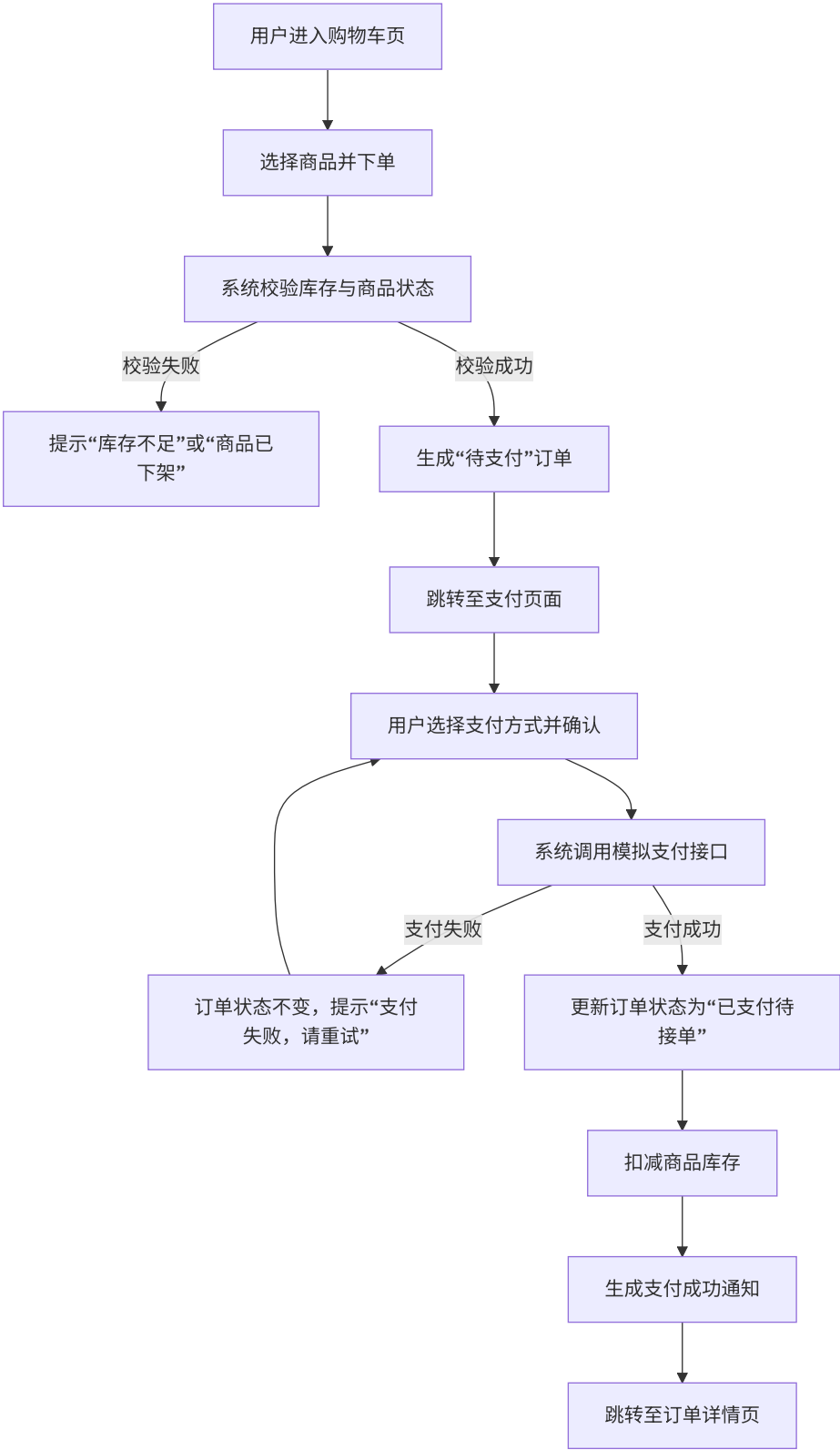
本系统采用前后端分离的四层架构。基于概要设计，将系统进一步细分为以下核心子系统（或称为微服务模块，在单体架构中为独立的功能模块）。每个子系统内部保持高内聚，子系统之间通过RESTful API进行通信，实现低耦合。

以下是核心业务流程的UML活动图与顺序图描述，以“用户下单支付”流程为例。

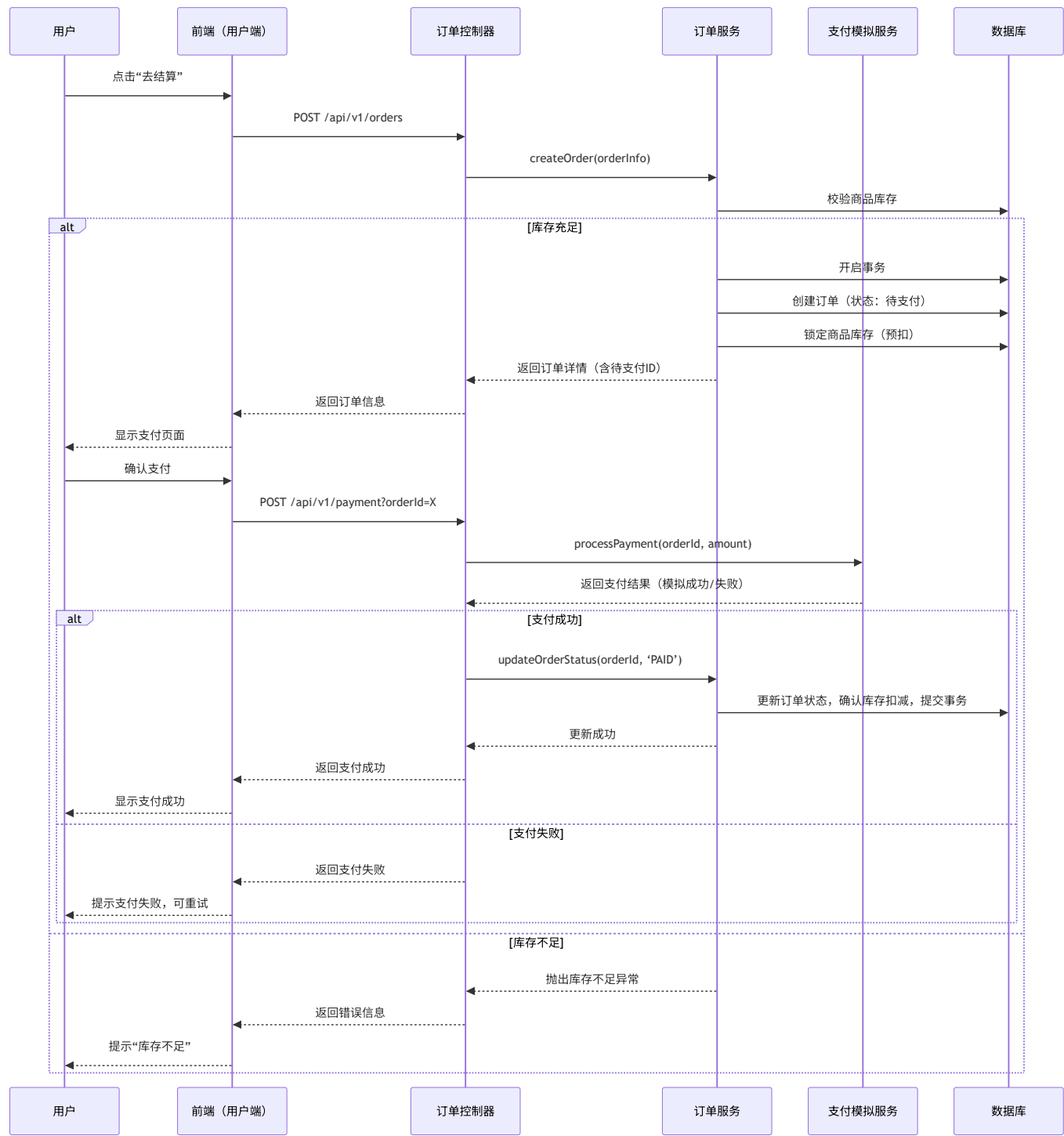
子系统划分：

- 认证授权子系统：负责用户登录、JWT Token的颁发与校验。

活动图示例：“用户下单支付”流程

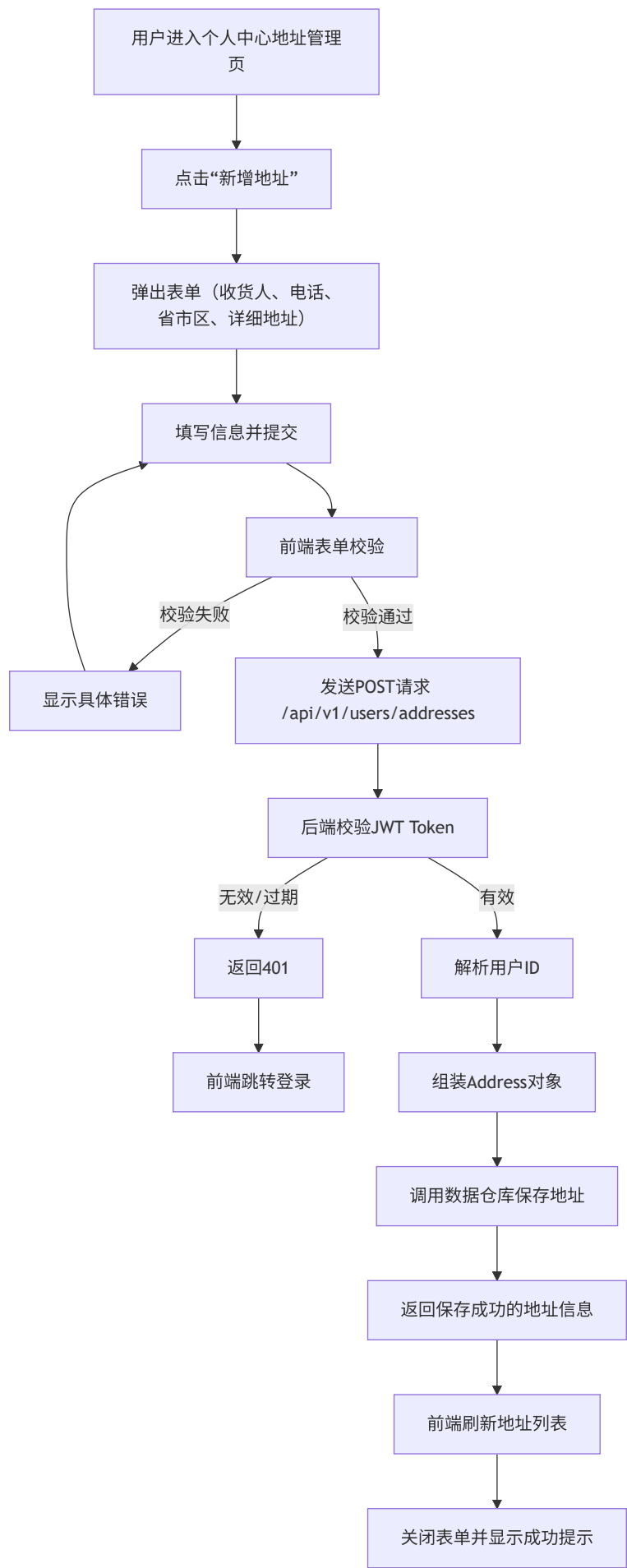


顺序图示例：“用户下单支付”流程

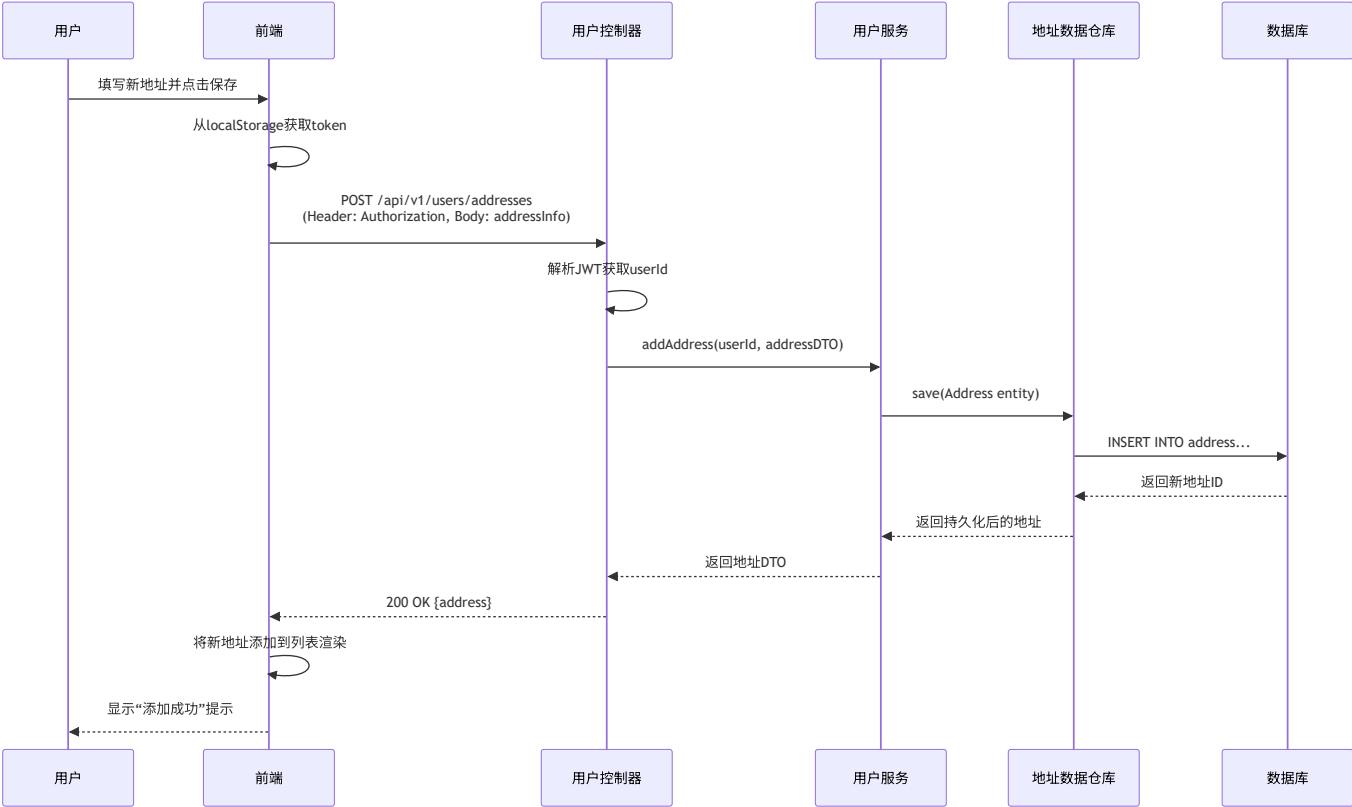


2. 用户管理子系统：管理用户基本信息、收货地址（以“添加收货地址”为例）

活动图：添加收货地址

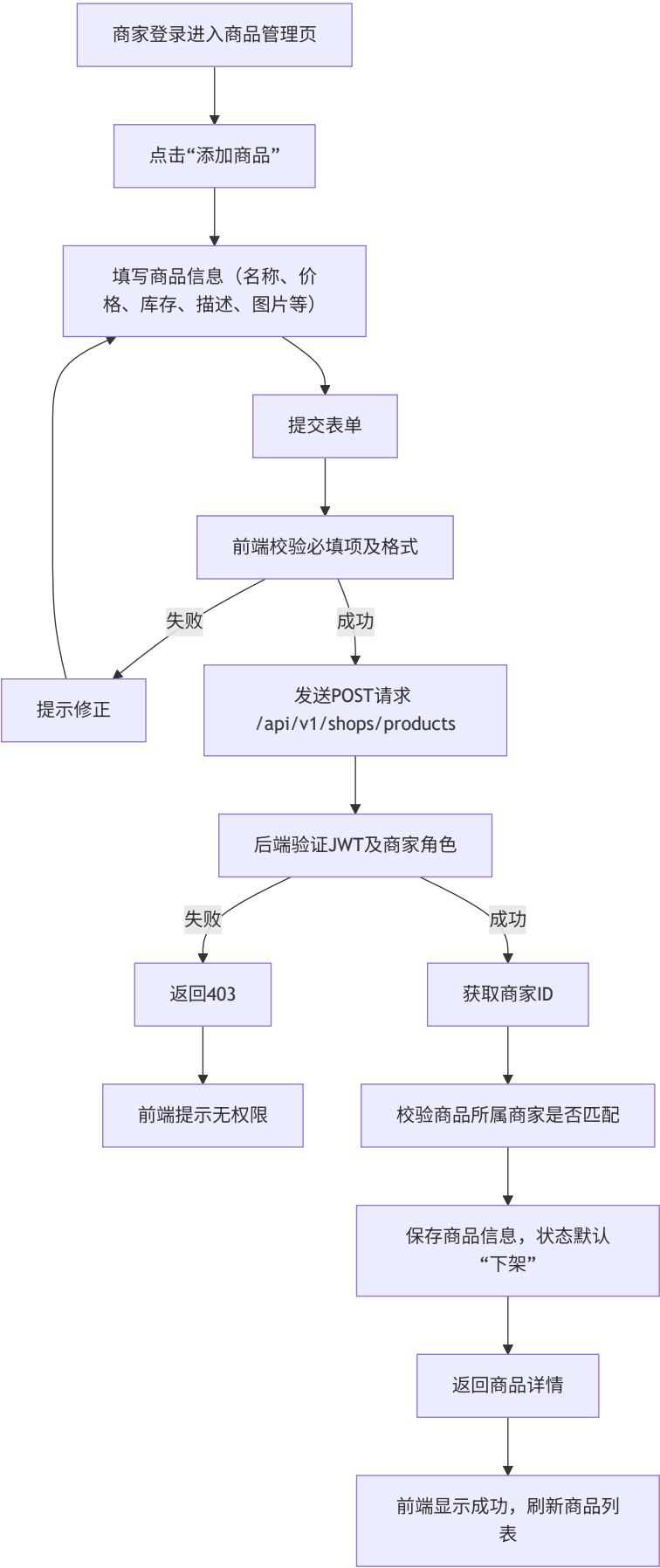


顺序图：添加收货地址

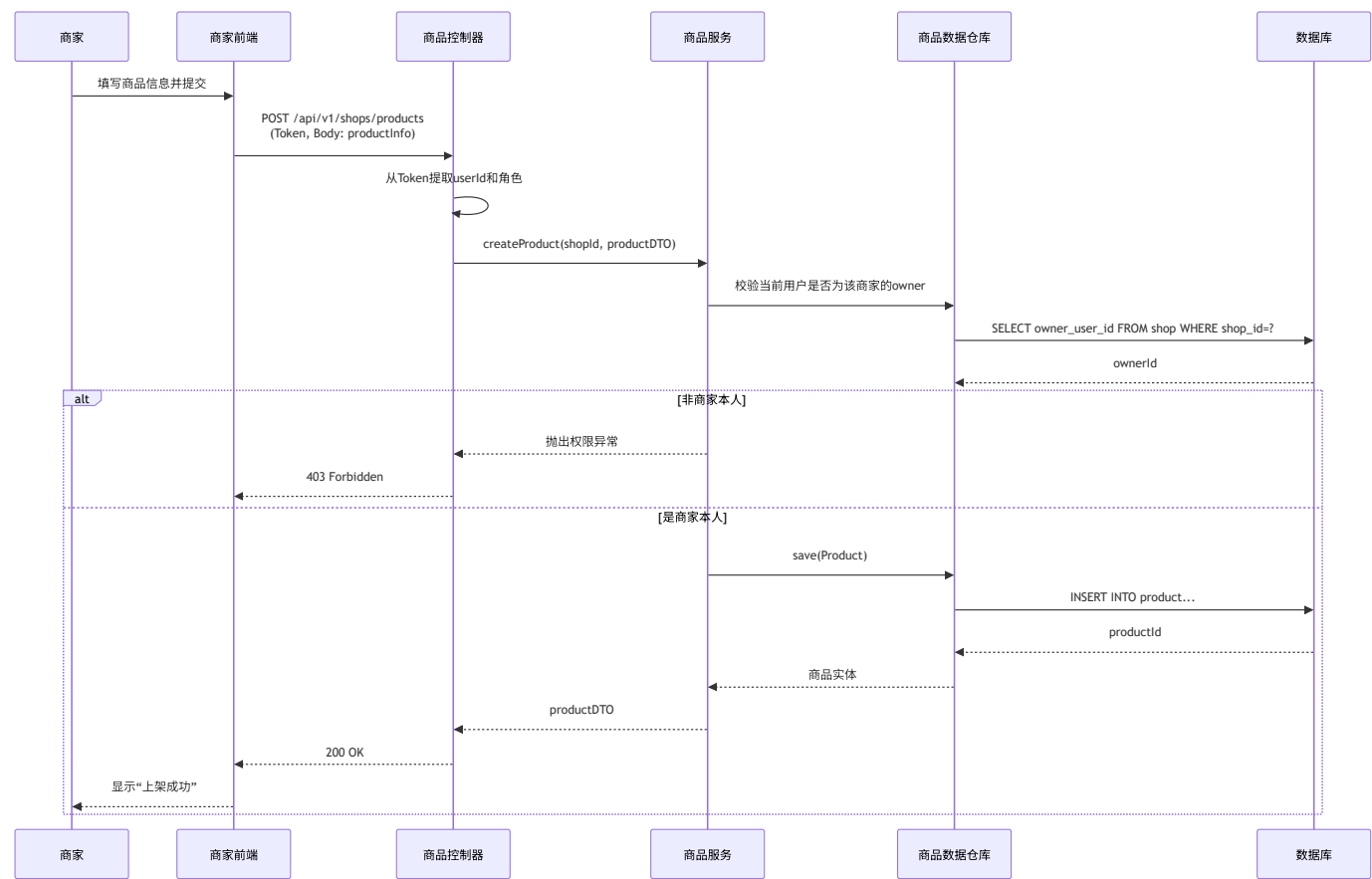


3. 商家管理子系统：管理商家入驻信息、分类、商品（以“商品上架”为例）

活动图：商家上架商品

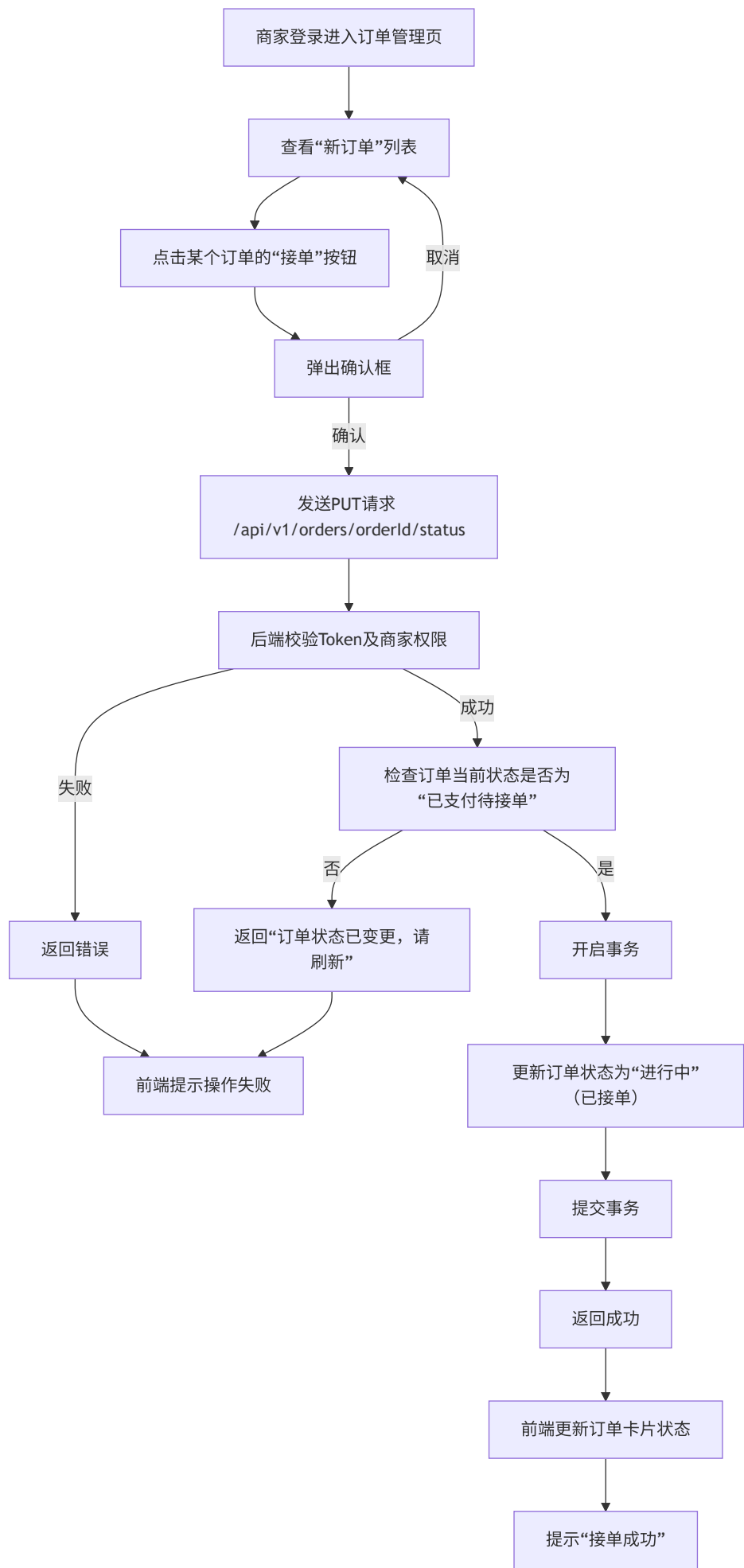


顺序图：商家上架商品

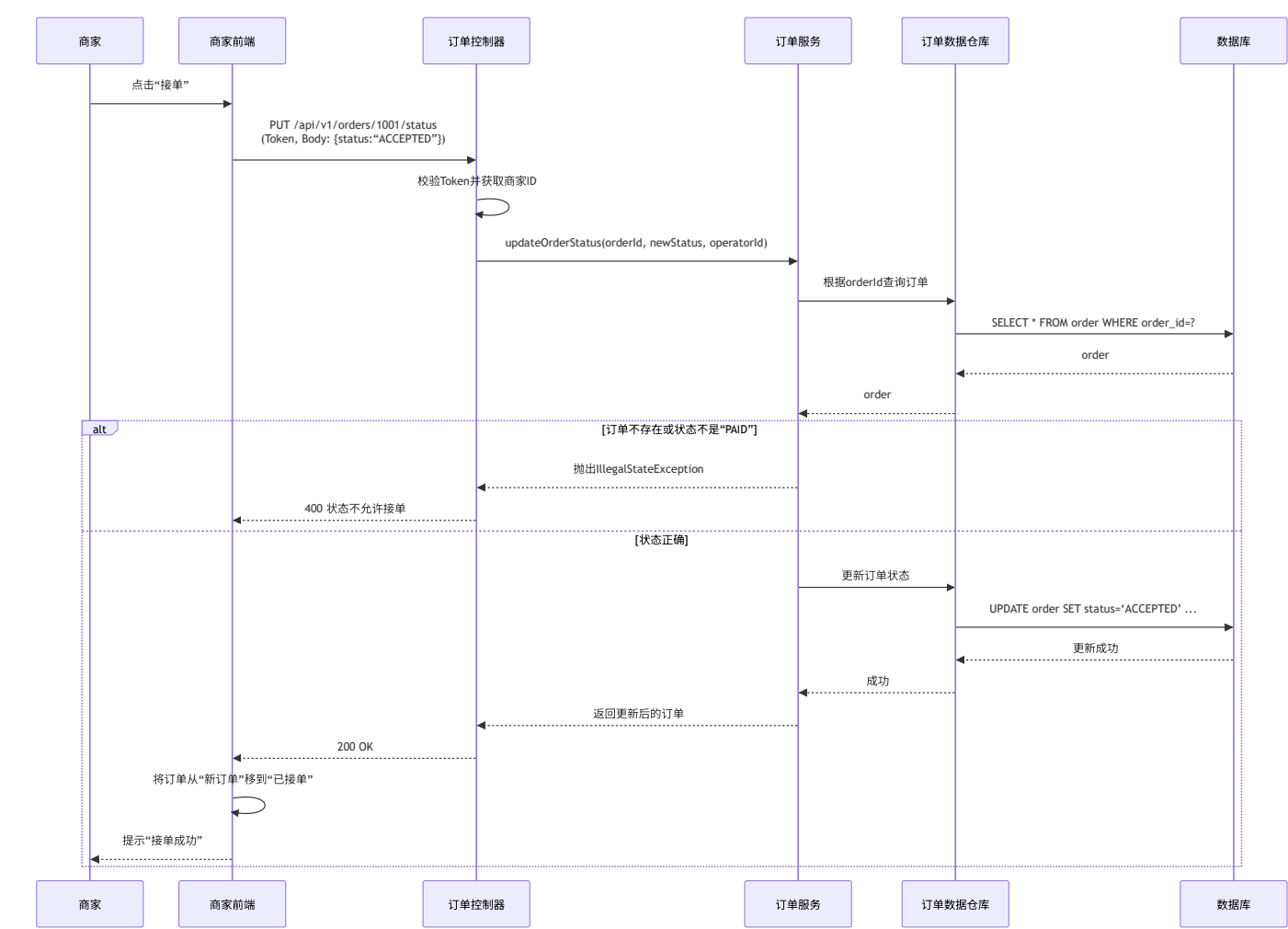


4. 订单管理子系统：处理购物车、订单生成、订单状态流转（以“商家接单”为例）

活动图：商家接单流程

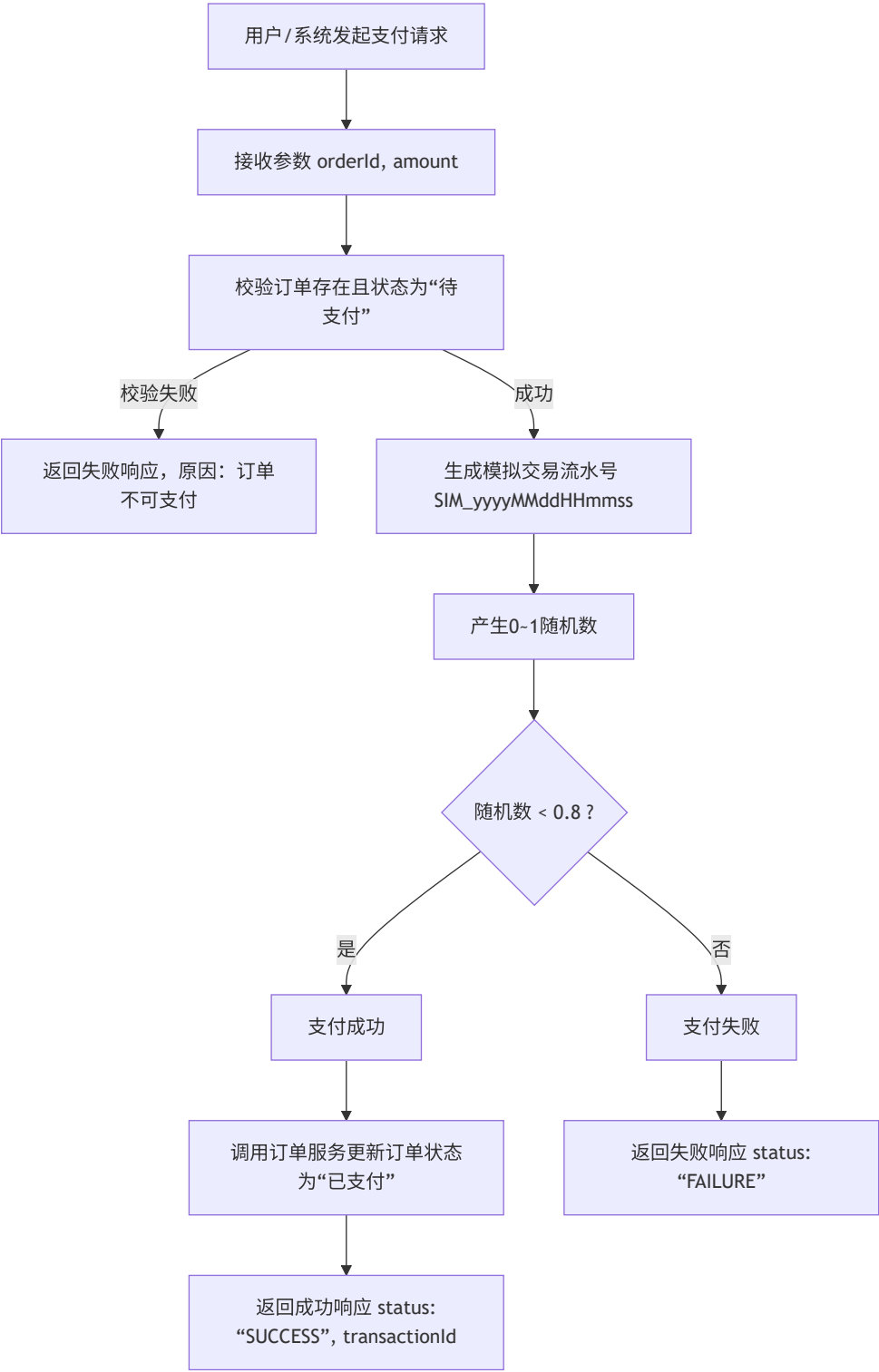


顺序图：商家接单

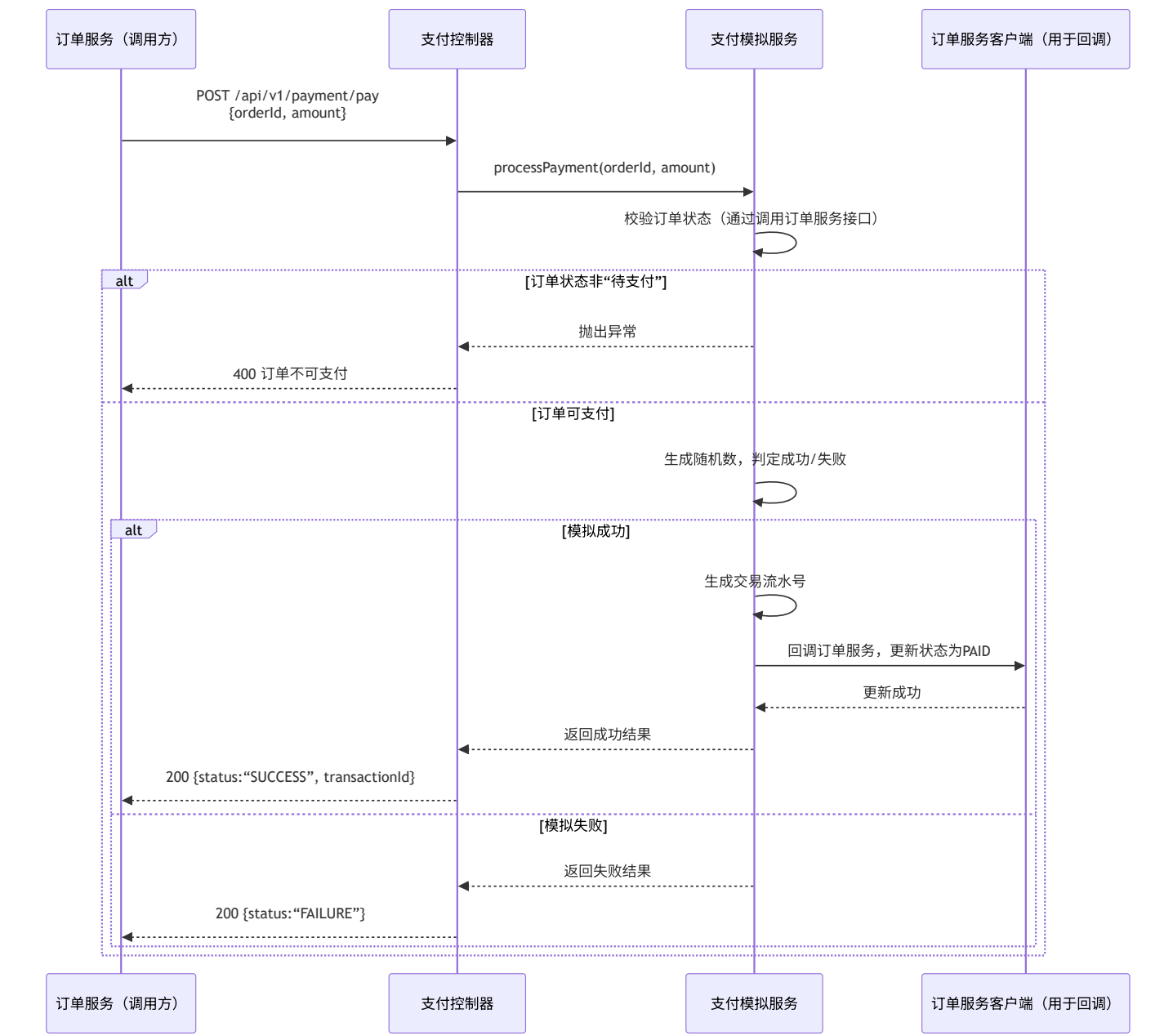


5. 支付模拟子系统：提供模拟支付接口

活动图：模拟支付

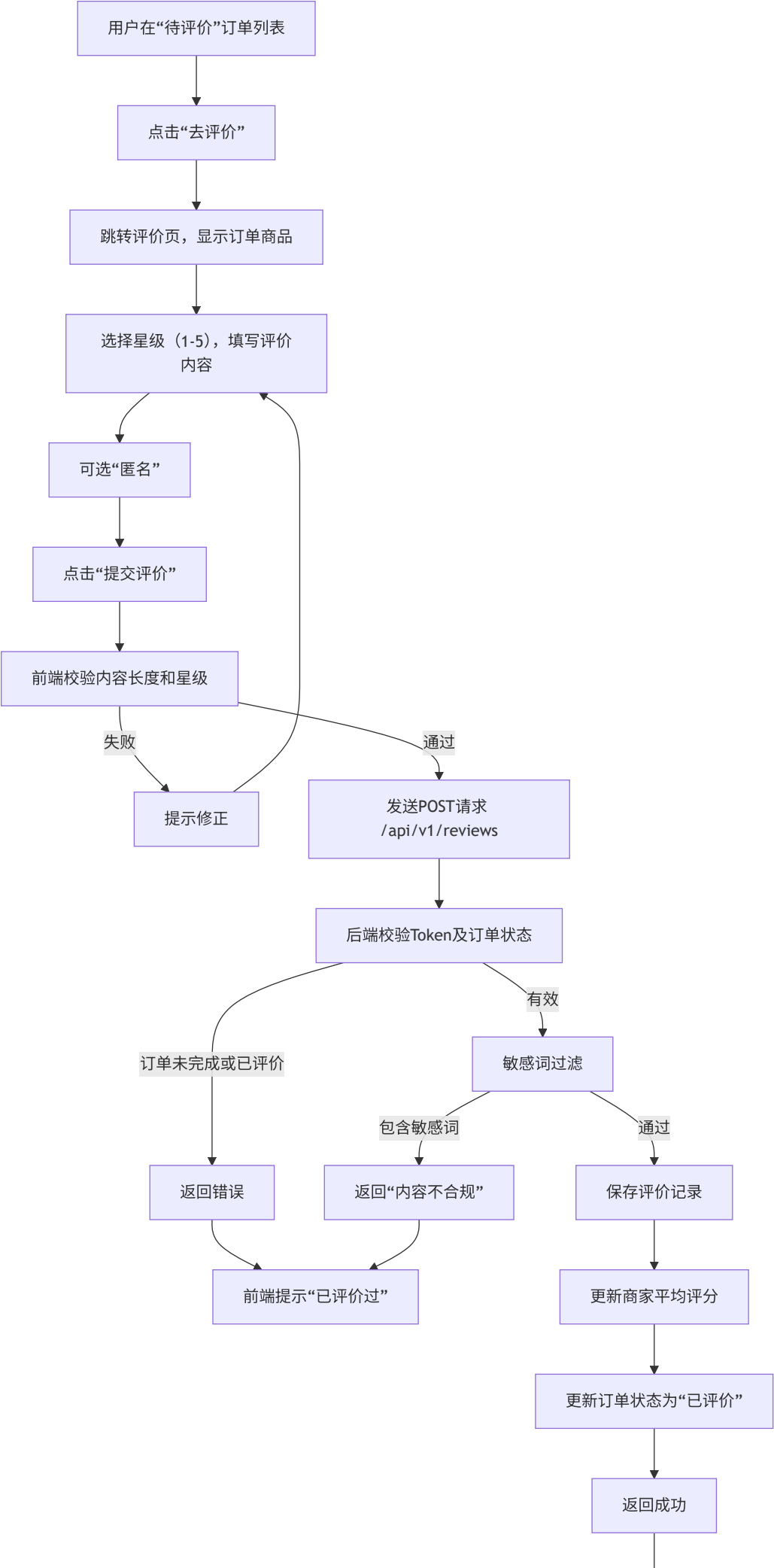


顺序图：模拟支付



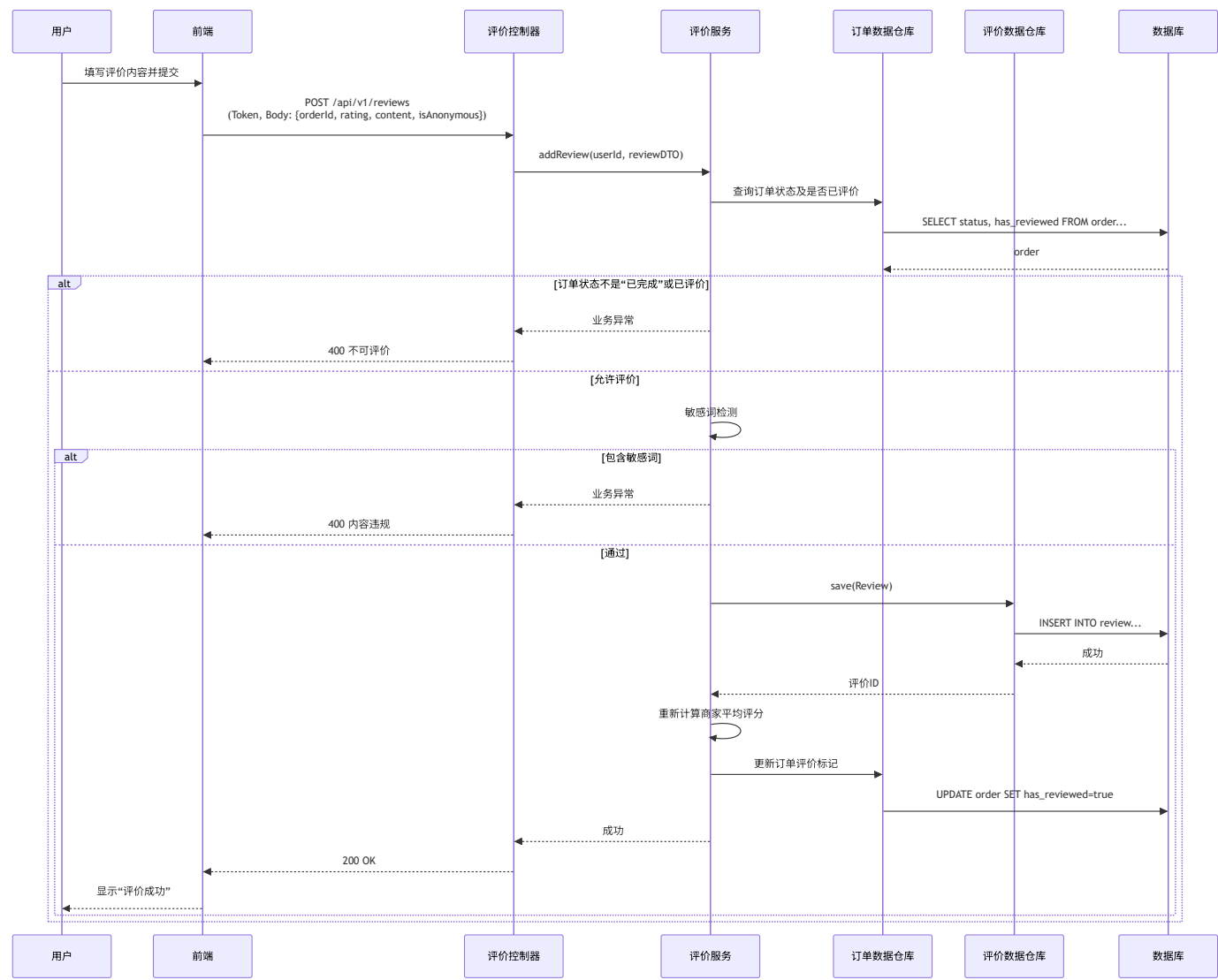
6. 评价管理子系统：处理用户评价及商家回复（以“用户发表评价”为例）

活动图：用户发表评价

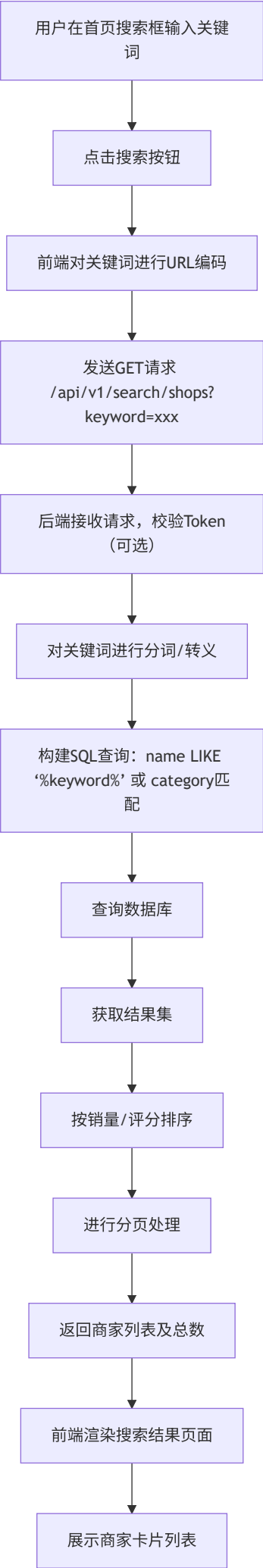


前端跳转回订单列表，显示已评价

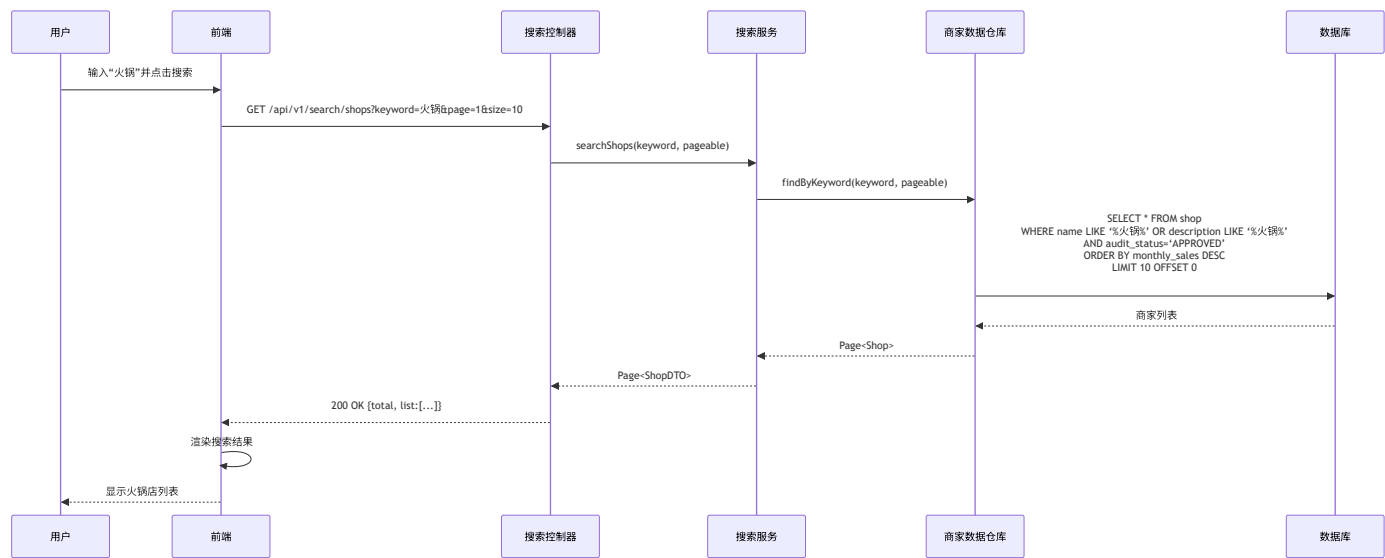
顺序图：用户发表评价



7. 搜索推荐子系统：实现商家和商品的搜索功能（以“搜索商家”为例）

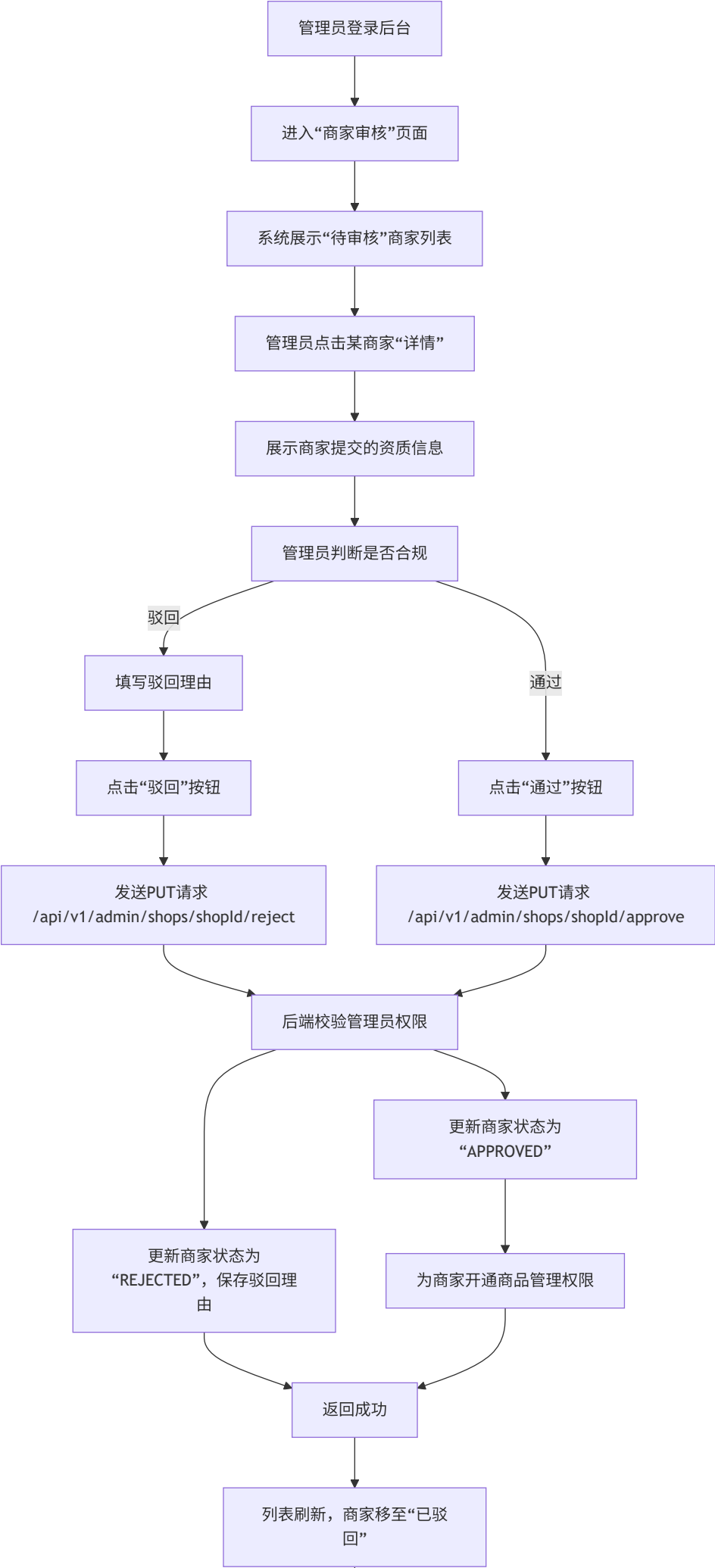


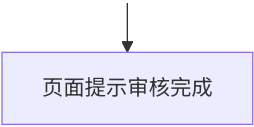
顺序图：搜索商家



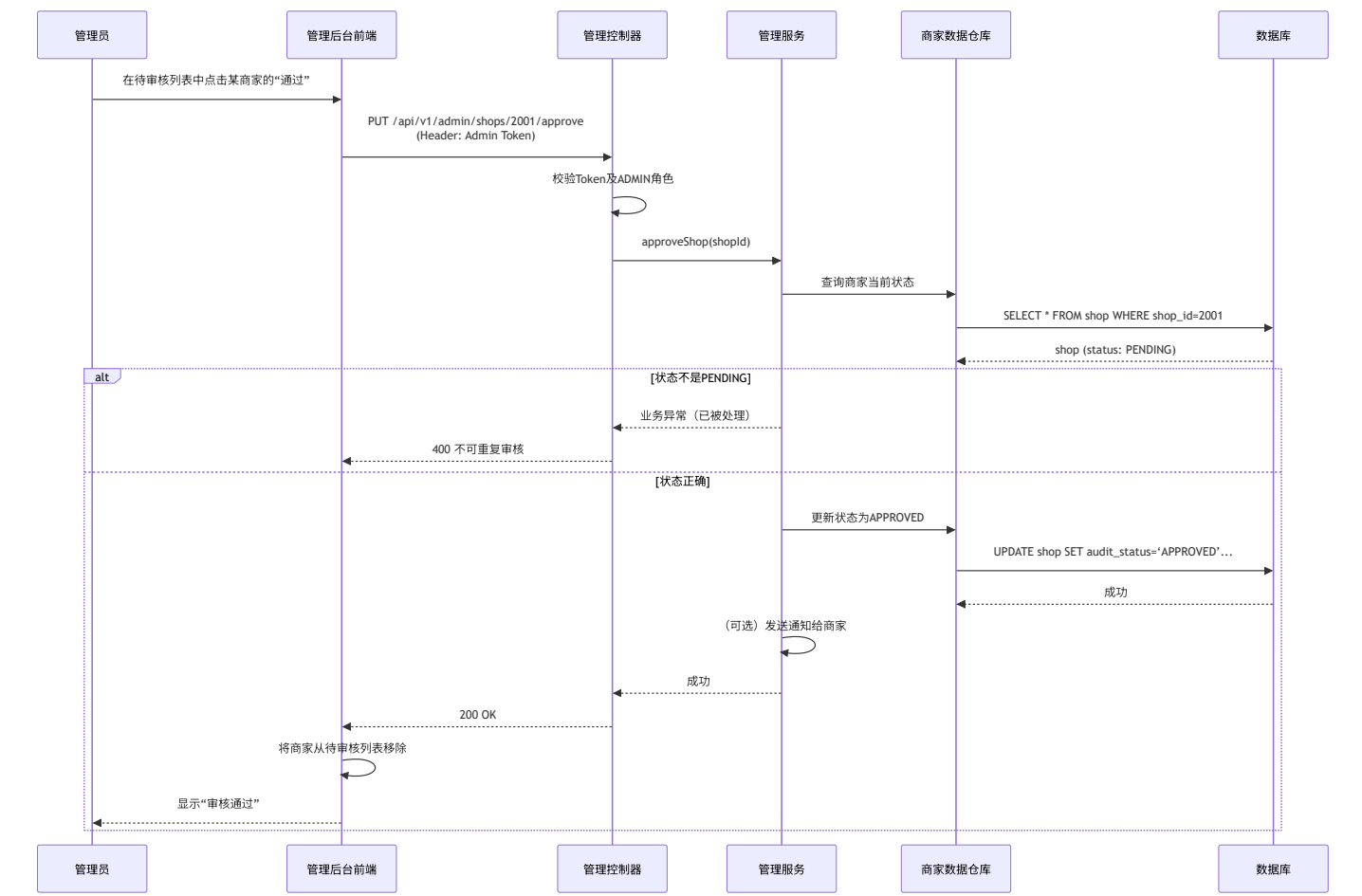
8. 后台管理子系统：为管理员提供审核、用户管理、数据统计（以“管理员审核商家入驻”为例）

活动图：审核商家入驻





顺序图：审核商家入驻



3 功能模块设计

本节对核心功能模块进行详细设计，描述其输入、处理逻辑和输出。

模块编号	模块名称	输入	处理	输出
M-AUTH-01	用户注册	{phone, password, nickname}	1. 校验手机号格式及是否已存在。 2. 对密码加盐（Bcrypt）哈希。 3. 创建新用户记录，角色默认为 <code>USER</code> 。 4. 返回注册成功信息。	E
M-AUTH-02	用户登录	{account(phone/email), password}	1. 根据账号查找用户。 2. 用Bcrypt验证密码。 3. 验证通过后，生成包含 <code>userId</code> 和 <code>role</code> 的JWT Token。 4. 返回Token和用户基本信息。	J
M-ORDER-01	创建订单	{userId, shopId, items[{productId, quantity}], addressId, remark}	1. 验证用户状态及地址有效性。 2. 遍历商品项，校验价格、库存、上架状态。 3. 计算总金额。 4. 开启数据库事务。 5. 生成唯一订单号 <code>order_sn</code> ，创建订单（状态：待支付）。 6. 创建订单行项目快照。 7. 乐观锁扣减商品库存。 8. 提交事务。	P

模块编号	模块名称	输入	处理	备注
M-PAY-01	模拟支付	{orderId, amount}	1. 校验订单状态为“待支付”。 2. 生成模拟交易流水号 SIM_yyyyMMddHHmmss。 3. 按预设概率（80%成功）随机返回支付结果。 4. 支付成功后，调用订单服务更新状态。	支付
M-SHOP-01	商家搜索	{keyword, categoryId, sortBy(销量/评分/距离), page, size}	1. 解析搜索条件。 2. 构建动态SQL查询。 3. 从数据库中检索匹配的商家列表。 4. 根据排序规则进行排序。 5. 进行分页处理。	商家
M-REVIEW-01	添加评价	{userId, orderId, rating, content, isAnonymous}	1. 校验订单状态为“已完成”且未被评价。 2. 敏感词过滤（基础关键词匹配）。 3. 创建评价记录，若匿名则隐藏用户昵称。 4. 更新关联商家的 average_rating。 5. 订单状态变更为“已评价”。	评价
M-ADMIN-01	数据统计	{startDate, endDate}	1. 查询指定时间范围内的订单总数和总交易额。 2. 查询指定时间范围内的新增用户数。 3. 查询各商家订单量排行榜。 4. 组装数据返回给前端仪表盘。	统计

4 界面设计

4.1 外部界面设计

系统外部界面为用户浏览器，所有功能均通过Web页面进行交互。后端提供统一的RESTful API（前缀 /api/v1/ ），前端通过AJAX/Fetch请求获取JSON数据并动态渲染页面。

4.2 内部界面设计

各功能模块间通过清晰的接口进行调用，其主要调用关系如下：

- 表现层 (Controller) 接收前端请求，进行参数校验后，调用 业务层 (Service) 的对应方法。
- 业务层 (Service) 封装核心业务逻辑，根据需要调用 数据访问层 (Repository/DAO) 或其他Service。
- 数据访问层 (Repository/DAO) 封装对数据库的CRUD操作。

示例接口定义：

- 订单服务接口 (IOrderService):
 - Order createOrder(CreateOrderDTO dto) - 创建订单
 - Order updateOrderStatus(Long orderId, String status) - 更新订单状态

4.3 用户界面设计

系统采用响应式布局，主色调为活力橙（ #FF7E36 ）和清爽白/灰色，营造生活服务的亲切感。

4.3.1 用户端页面

- 首页：顶部为搜索栏，下方为轮播图，主体为商家分类图标区和“附近热门商家”信息流列表。
- 商家详情页：顶部展示商家信息（名称、评分、月售、公告等），中间区域以Tab形式展示“点餐”(外卖商品列表，包含加购按钮)和“团购”(团购商品列表，包含“抢购”按钮)，底部为固定的“购物车”浮层。
- 订单页：顶部Tab（全部、待支付、待收货、待评价、退款/售后），下方为订单卡片列表，每个卡片清晰展示订单号、商家名、商品快照、实付金额和当前状态操作按钮（如“去支付”、“确认收货”、“去评价”）。

4.3.2 商家端页面

- 商品管理页：提供商品列表，支持按上下架状态筛选。每个商品项后跟“编辑”、“下架/上架”按钮。顶部有“新增商品”按钮，点击弹出包含名称、价格、库存等字段的表单模态框。
- 订单处理页：Tab分为“新订单”、“已接单”、“已完成”。新订单卡片显示配送地址、订单明细，并有“接单”、“拒单”按钮。

4.3.3 管理员端页面

- 统计仪表盘：使用ECharts图表库展示数据趋势，包括折线图（近7日交易额趋势）、饼图（订单类型分布）、柱状图（商家销量Top10）。

5 异常处理设计

5.1 出错信息管理

- 前端：对所有用户输入进行本地校验（如手机号格式、必填项），第一时间给出明确提示。对于后端返回的错误，在页面顶部Toast或对应字段旁显示统一格式的错误信息，如“手机号已注册”、“网络开小差了，请稍后再试”。
- 后端：设计 GlobalExceptionHandler 全局异常处理器，将不同类型的异常（如 MethodArgumentNotValidException，BusinessException，RuntimeException）统一封装为标准的错误响应格式 {“code”: 500, “msg”: “具体错误描述”}，不向客户端暴露堆栈信息。

5.2 故障预防与补救

- 订单一致性：在“下单-扣库存”流程中使用数据库事务，确保操作原子性。对商品库存使用乐观锁，防止高并发下超卖。若支付环节失败，订单状态保持“待支付”，用户可在有效期内重试。
- 接口幂等性：关键接口（如创建订单）通过前端传递唯一标识（Idempotent-Key）实现幂等性，防止网络重复提交导致重复下单。
- 数据备份：定期（课程演示前）通过 mysqldump 工具备份数据库。
- 日志记录：使用SLF4J记录关键操作（如支付、审核）、系统异常和用户登录行为，日志保存30天，便于故障追溯。

5.3 系统维护设计

提供基于角色的后台管理功能，管理员可：

- 查看系统健康状态（如数据库连接池状态、内存使用情况）。
- 动态开关“维护模式”，维护期间前端所有非管理员请求将返回503，并提示“系统维护中”。

6 性能优化和安全设计

- 性能优化
 - 数据库索引：为 Order 表的 user_id，shop_id，status，Shop 表的 category_id，audit_status，User 表的 phone 等高频查询字段创建索引。
 - 数据缓存：对于首页商家列表、分类等变化不频繁的数据，可使用本地缓存（Caffeine）或Redis缓存，设置合理过期时间（如10分钟），降低数据库压力。
 - 分页查询：所有列表查询（订单、商家、商品）均实现后端分页，避免一次性加载大量数据。
- 安全设计
 - 认证与授权：
 - 登录认证：用户密码使用BCrypt加盐哈希存储。JWT Token设置合理有效期（如7天）。
 - 接口鉴权：实现全局拦截器，对所有 /api/v1/** 下的请求验证JWT Token的有效性。利用Spring Security或自定义注解（@PreAuthorize）在方法级别校验用户角色（USER, SHOP, DELIVERY, ADMIN），防止横向越权和垂直越权。
 - 数据安全：
 - 传输安全：生产环境建议启用HTTPS，确保数据传输加密（实验环境为HTTP）。
 - 防SQL注入：使用ORM框架（如MyBatis的#{ }）进行参数化查询，严禁使用字符串拼接SQL。
 - 敏感数据脱敏：前端展示用户手机号时，中间四位用 * 替换（如 138****1234）。
 - 输入校验：前后端双重校验用户输入，对评价内容、商家名称等字段进行XSS（跨站脚本攻击）过滤。

7 项目测试计划

序号	测试类型	对应部分	测试内容	计划测试时间
1	单元测试	后端服务层	1. createOrder 方法：正常流程、库存不足、商品不存在。 2. processPayment 方法：成功/失败逻辑。 3. calculateRating 评分更新算法正确性。	编码阶段（第3-4周）
2	接口测试	RESTful API	1. 用户注册/登录接口。 2. 创建/取消/查询订单接口。 3. 商家添加/下架商品接口。 4. 管理员审核商家接口。 5. 验证JWT Token鉴权失效。	编码完成后（第4周）
3	集成测试	前后端交互	1. 完整业务流程测试：搜索商家->加购->下单->支付->收货->评价。 2. 商家端订单处理流程。	前后端联调阶段（第5周）

序号	测试类型	对应部分	测试内容	计划测试时间
			3. 配送员状态更新流程。	
4	系统测试	全系统	1. 多角色（用户/商家/配送/管理员）功能覆盖。 2. 异常场景测试：支付失败、取消订单、退款申请。 3. 简单并发测试：模拟10个用户同时下单同一商品。 4. 浏览器兼容性测试（Chrome, Edge）。	系统测试阶段（第6周）